

ME 5245 Project
Position and Speed control of DC motor system in Simulink
Real Time Target and Arduino

By

Sri harish Om Singaravelan Poornima

Circuit Diagram:

The circuit diagram includes the DC motor, Arduino UNO, H-bridge, potentiometer, and power supply. The motor's power terminals are connected to the H-bridge outputs, while the control inputs of the H-bridge link to specific digital pins on the Arduino. The potentiometer is connected to an analog input on the Arduino to provide a variable voltage signal. It's crucial to reference the Tinkercad link for a visual representation and ensure all connections are accurately depicted for replication and troubleshooting.

The most important connections are Arduino pins, A0, 2, 3, 5, 6, and 10. The potentiometer connects to analog input A0 as it reads a voltage value from 0-1023 representing 0-5V. This signal is used to control the output signal of pin 10, which supplies a PWM signal to the H-bridge. This tells the H-bridge how much power from the 7.5V's of the battery pack it should supply to the motor. Pins 2 and 3 read the encoders from the motor and 5 and 6 tell the motor what direction to spin. Here pins 5 and 6 were not used as the motor was directly powered using the battery pack.

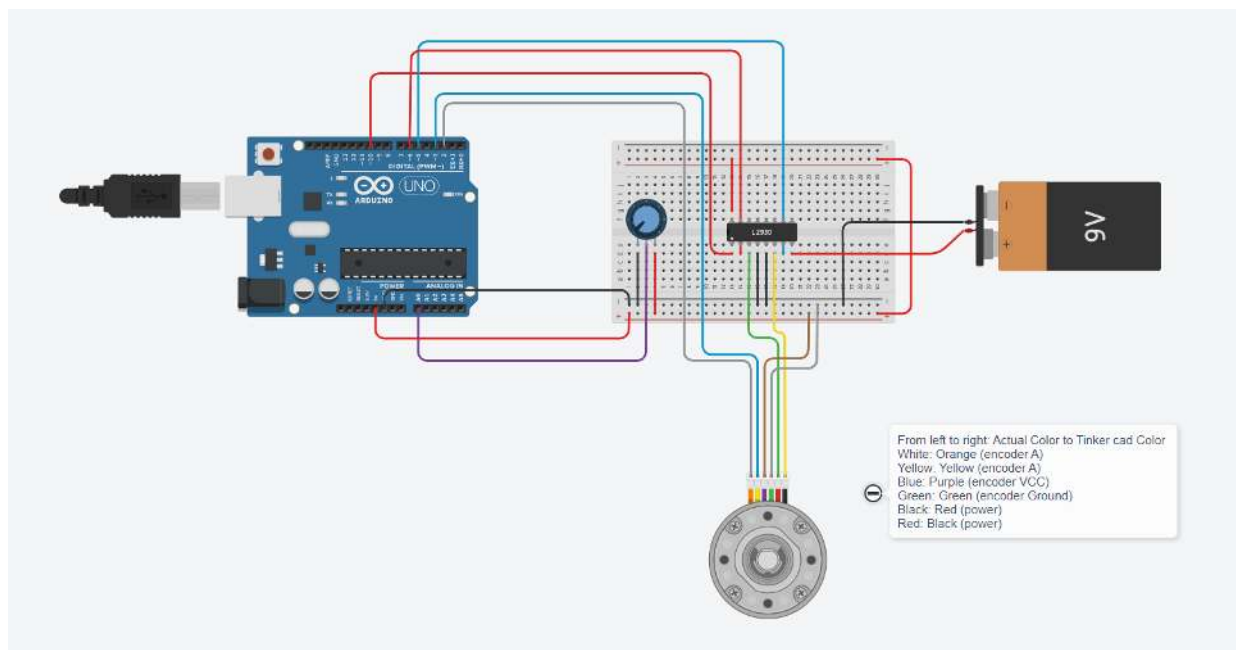


Figure 1: Circuit Diagram

Hardware Used:

1. Pololu 25D Metal Gearmotor
2. Arduino UNO
3. H-bridge (L293x)
4. Potentiometer
5. Power supply
6. Assorted cables and connectors
7. Breadboard

Encoder Reading:

An encoder is an electromechanical device that converts the angular position or motion of a shaft into digital signals. In this project, the encoder is used to measure the rotor's displacement and speed. The encoder generates pulses as the motor shaft rotates, and these pulses are counted in Simulink to determine the angular position of the rotor in degrees. The rotor speed is calculated by measuring the rate of change in the pulse count over time, which is then converted into revolutions per minute (RPM). The accuracy of the position measurement is limited by the encoder resolution, which defines the smallest detectable angular change, introducing an error in the position reading

Displacement Data:

For the position data it is multiplied by 360 degrees and divided by the pulse count multiplied by the gear ratio. The scope produced from this diagram is shown in Figure 6. Since the encoder doesn't have any limit for data collection, we see in fig.3 that the displacement angle incrementally increases.

FORMULA USED:

$$\text{Position in degrees: } \frac{(\text{Encoder Count} * 360)}{(48 * 172)}$$

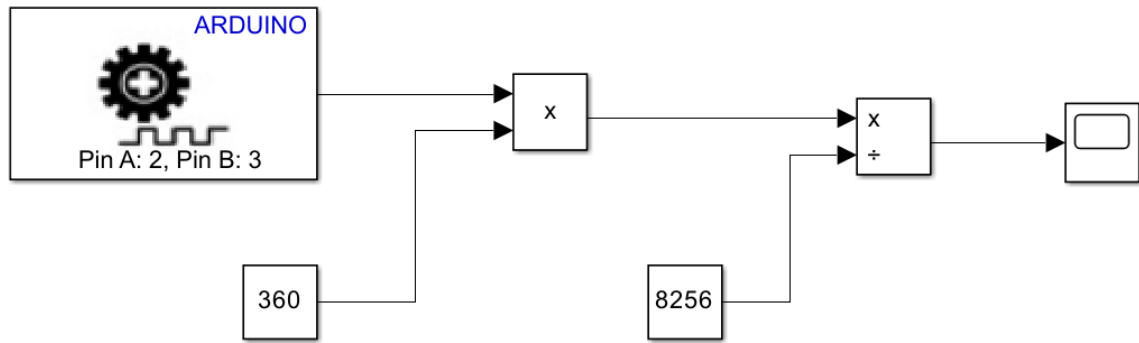


Figure 2: Simulink Position Diagram

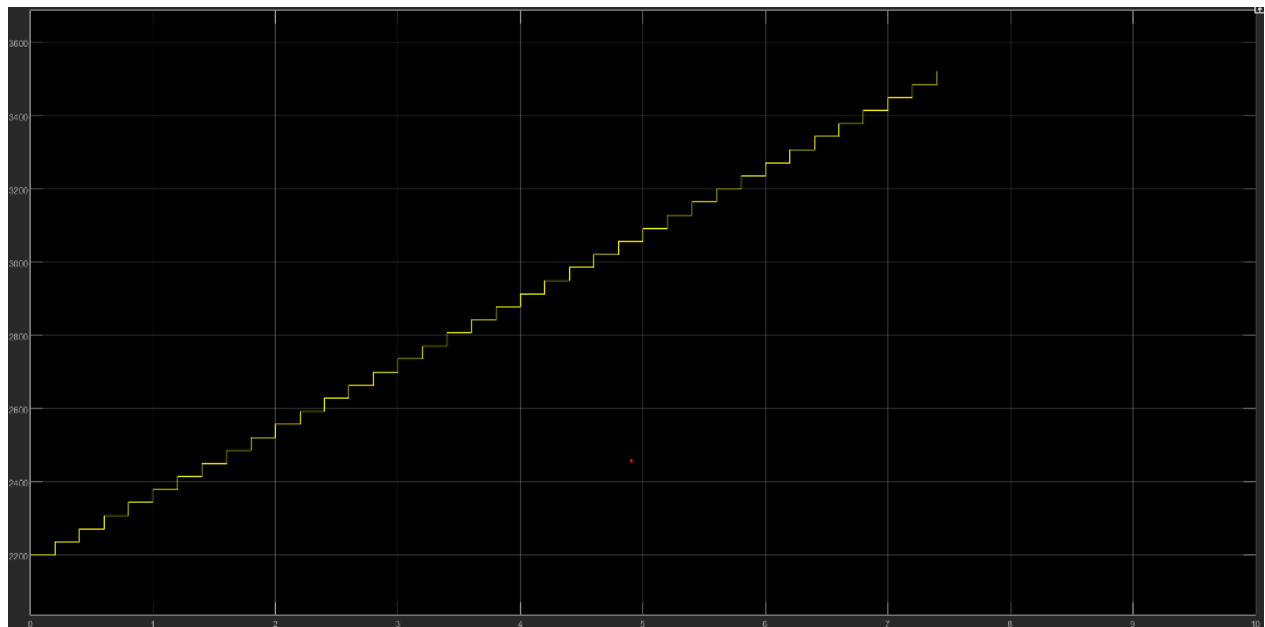


Figure 3: Position Data Scope

Simulink for Velocity:

The velocity is found by splitting the signal from the Displacement so that you have the count prior and the current count, subtracting the current from the previous, and dividing it by the time step, and multiplying by 60 seconds per minute to obtain revolutions per minute. To ensure an accurate reading. The encoder value settles at a value of 31. The speed value is larger than no load speed due to the voltage value of the battery.

FORMULA USED:

$$\text{Speed} = (Position_1 - Position_2) * \frac{60}{0.01}$$

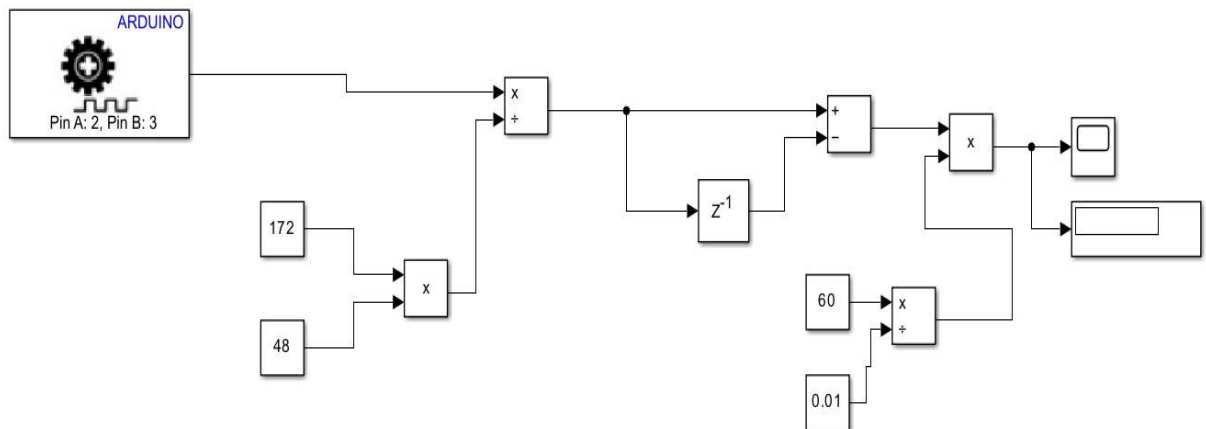


Figure 4: Simulink Velocity Diagram

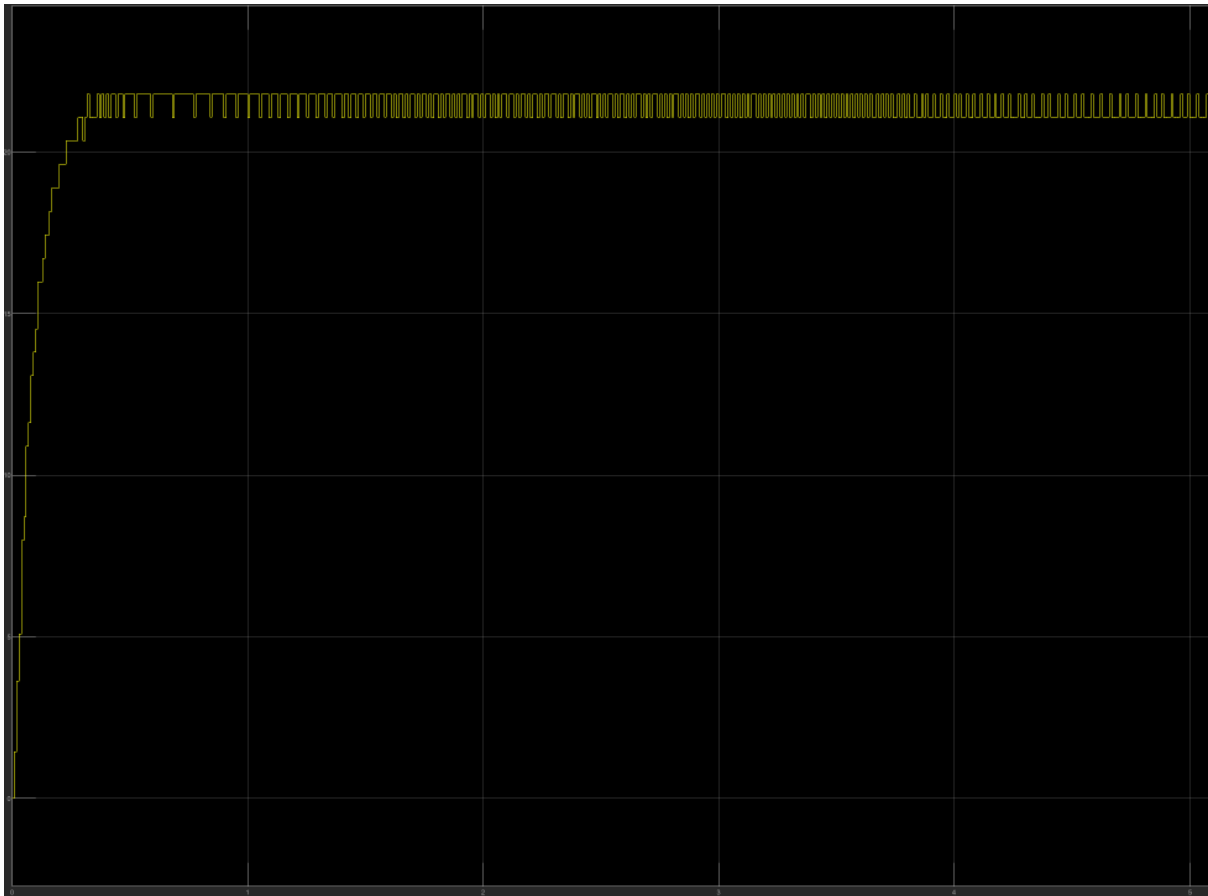


Figure 5: Velocity Scope

Position Error:

Due to the resolution of the encoder and the gear ratio of the motor, in degrees the smallest reading, and thus the error of the encoder is simply equation 1 at 1 encoder count.

So the position error is equal to:

$$\frac{360}{172 \cdot 48} = 0.0436$$

Potentiometer Signal :

Potentiometer signal is an analog signal which arises as a result of change in current as a result of change in current this analog value can be converted to PWM signal later for speed control. This signal can be extracted by using the Analog input block in Simulink and specifying the pin number as A0. The potentiometer was randomly changed to get an increase and a decrease in the signal shown in figure 10.

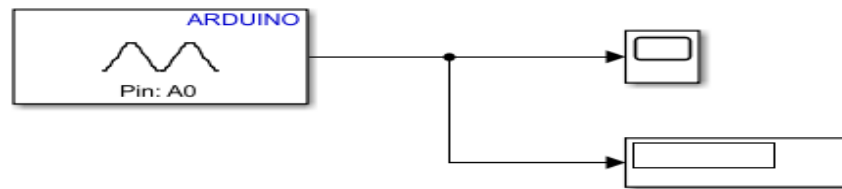


Figure 9: Potentiometer Reader

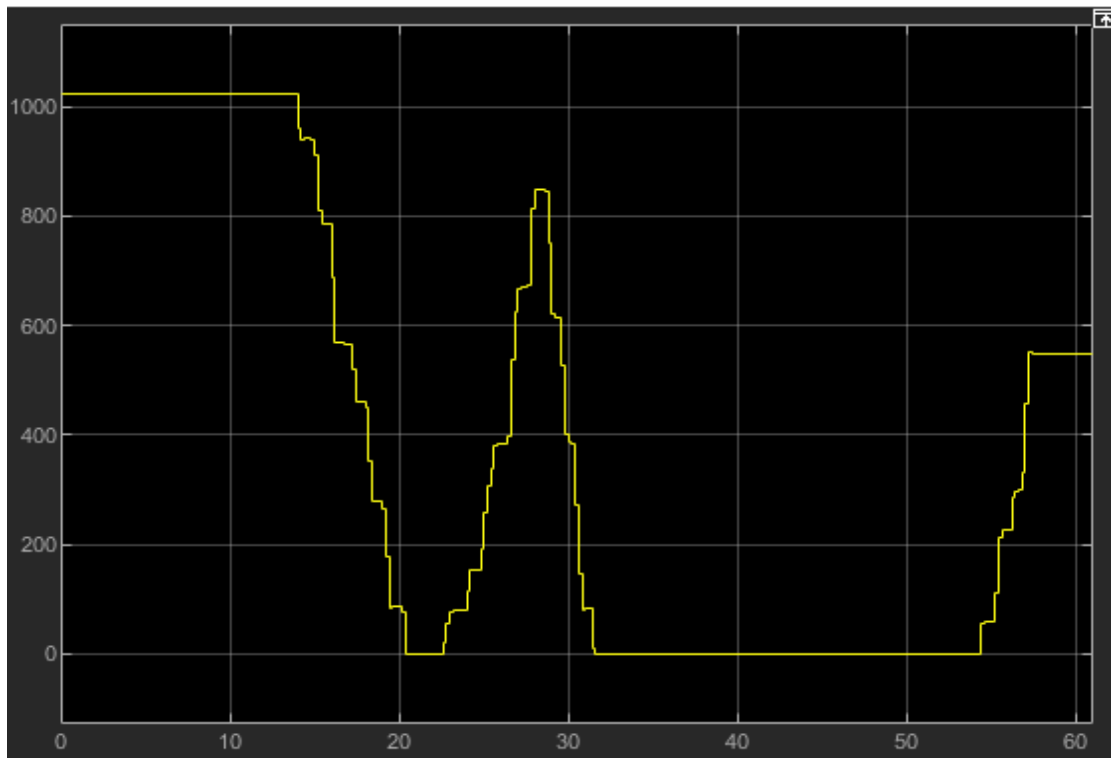


Figure 10: Potentiometer Reading

Circuit Diagram

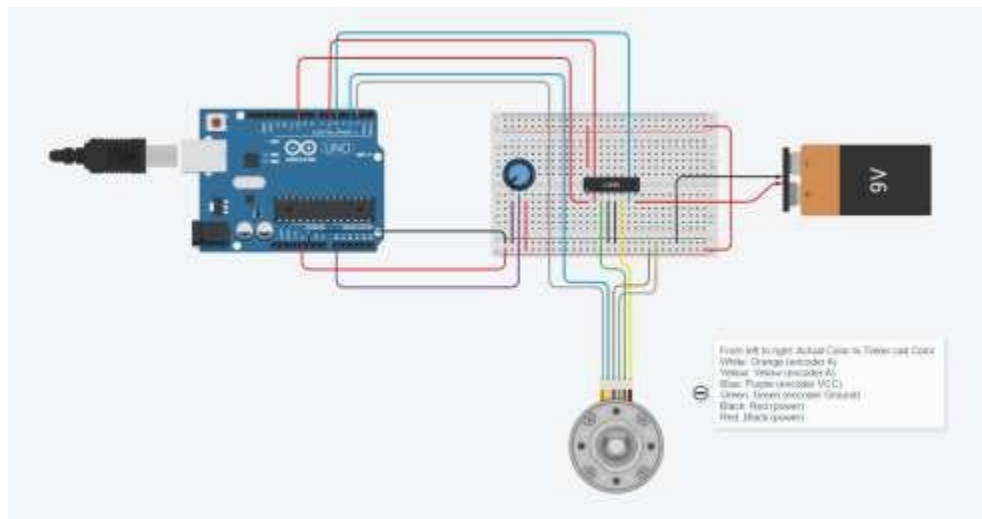


Fig 1

Open Loop Hardware Experiments

The Simulink model for controlling the speed of the DC motor is given below in Figure 2. This model directly interacts with the open-loop hardware system to allow for control of the position and speed of the motor as the potentiometer is turned. Varying the potentiometer is a way of adjusting the PWM duty cycle since the analogue signal from the potentiometer is converted to digital using the PWM block. Since the analog signals map from 0-5 V into variation form (0-255), the signal from the potentiometer needs to be converted from a scale of 0-1023 to a scale of 0-255. To read this in terms of duty ratio the 0-255 scale is then scaled to work with the 0-7.5V power supply to the motor and the 0-100 scale of the duty ratio.

Voltage Scale: Potentiometer Signal * 7.5/1023 (Equation 4)

Duty Ratio Scale: Potentiometer Signal * 100/1023 (Equation 5)

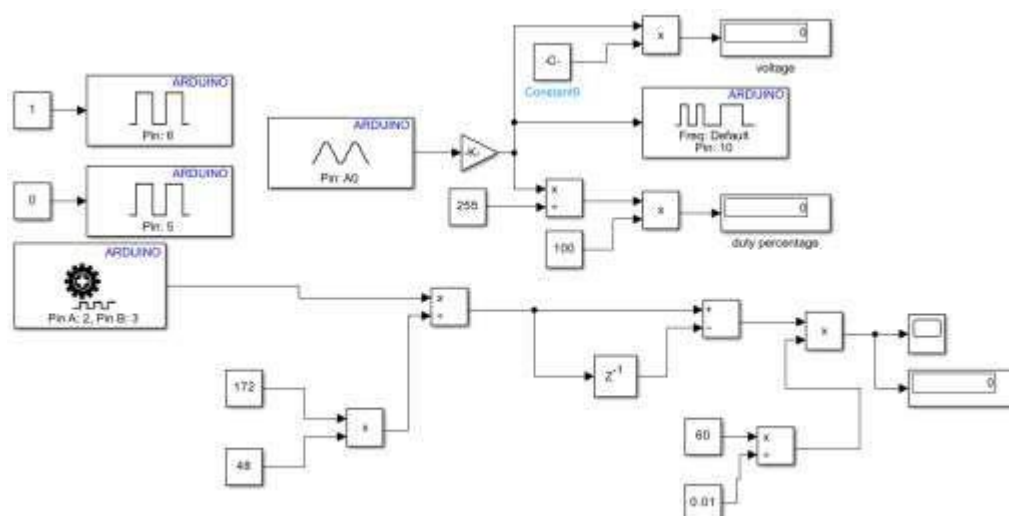


Figure 2: Simulink Duty cycle

We have added a real time scope display for duty ratio as well as its corresponding voltage. Additionally pins 5 and 6 now give signals to allow the motor to rotate, with a pulse of 1 sent to 6 and a pulse of 0 sent to 5. If these were reversed the motor would travel in the opposite direction. The motor was run with Duty Ratios 25, 50, 75, and 100, and the results of which were put into a graph shown in Figure 12. From this data an average back emf constant after the gear box and average dc gain were obtained.

Battery Voltage	Duty	RPM	Speed in Radian	DC Gain (RPM)	Back Emf Const.
7.522	100	31	3.246	4.121	0.2426
5.654	75	26.87	2.813	4.752	0.2104
3.772	50	20.37	2.133	5.403	0.1851
1.904	25	8	0.837	4.201	0.2380
	Average	21.56	2.25	4.61	0.2190

Table 1: Voltage

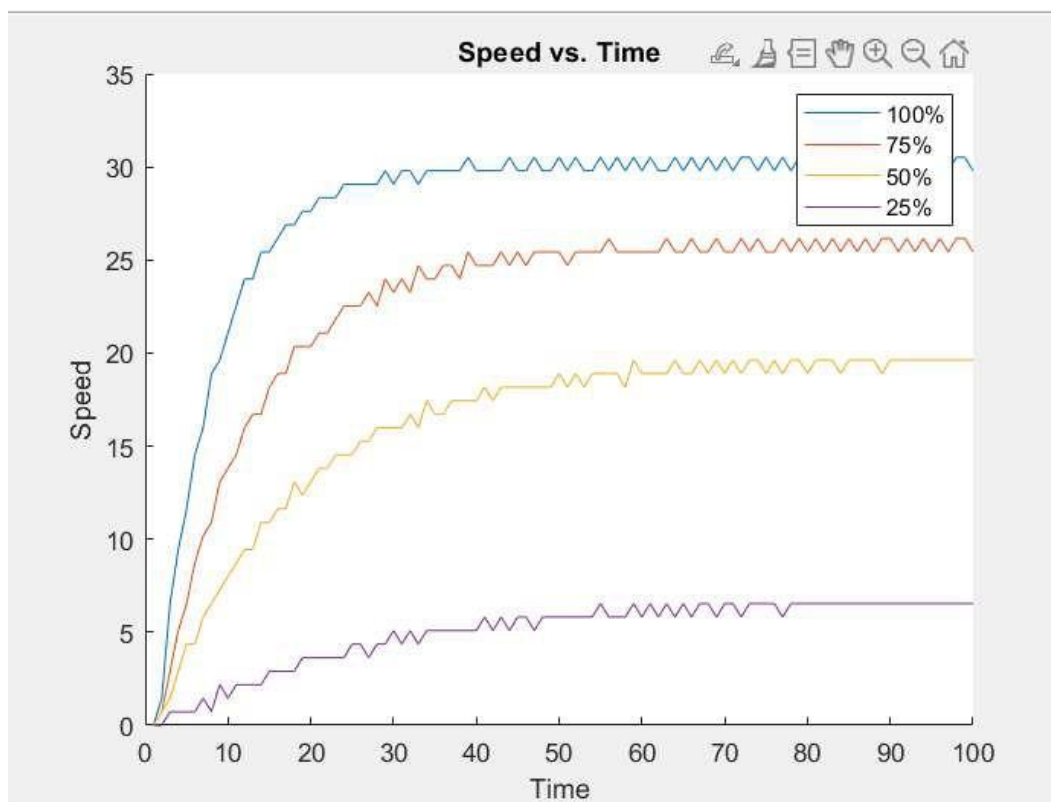


Figure 3: Speed in RPM vs Time in 0.01 seconds

Motor transfer function

The transfer function of the DC motor can be estimated from the response of the open loop hardware system. For the purposes of this project, it will be assumed the transfer function is first order and follows the equation:

$$G(s) = \frac{\Omega(s)}{V(s)} = \frac{K}{\tau s + 1} = \frac{\frac{K}{Rb + K^2}}{(\frac{RJ}{Rb + K^2})s + 1}$$

Where K in the first half of the equation is the DC Gain, τ is the time constant, R is the motor resistance, J is the motor inertia, b is the damping constant, and K in the second half is the back emf constant. DC Gain of the system is the value obtained when the system settles and we can find this by taking the limit as time tends to infinity for $\Omega(s)$. The DC gain we obtain will be the speed of the DC motor in (rad/sec)/Vin(s). From the plot we can observe that the maximum speed fluctuates from 31 to 32 rpm at 100 duty cycle.



Figure 4: Settling Time

The value of settling time is 0.493s after the transient period. We determined the value through cursor measurement of the scope shown in Figure 13. To find the time constant this value just needs to be divided by 4. Using the gain obtained in the previous, and plugging this into Equation 6, the equation for the transfer function is:

$$G(s) = \frac{4.61}{0.123s + 1}$$

Simulink Theoretical Approximation

To properly model the transfer function in Simulink, a PWM voltage controller block, DC motor block, h-bridge, gearbox, and two DC voltage sources are required. The DC Motor has an assumed inductance of 0, which may lead to inaccuracies compared to the real model. The PWM block is given a range of 0 to 5V and a frequency of 490.2 Hz, the frequency at which Arduinos run. The first voltage source starts off at 5V to mimic a 100% duty cycle. The gear box is given the gear ratio of the motor 171.79. The output of the gearbox is then read by a rotational speed sensor which The DC Motor meanwhile has 5 properties that need to be input. 0 for the inductance, the resistance is known to be 5 ohms, a motor inertia of $0.00976 \text{ kg}\cdot\text{m}^2$ after the gearbox, a back emf constant of 0.2190 V/rpm after the gear box, and the damping constant. These values needed to be converted so they act before the gear box and using this and equation 6 the damping constant can be calculated. To convert the motor inertia is divided by the gear ratio squared while the back emf is divided by just the gear ratio. These values come out to be $3.3072 \cdot 10^{-7} \text{ kg}\cdot\text{m}^2$ and 0.001275 V/rpm respectively. Using this and the DC Gain value, b comes out to be $5.788 \cdot 10^{-6}$. These values were then all put into the Simscape diagram shown in Figure 14.

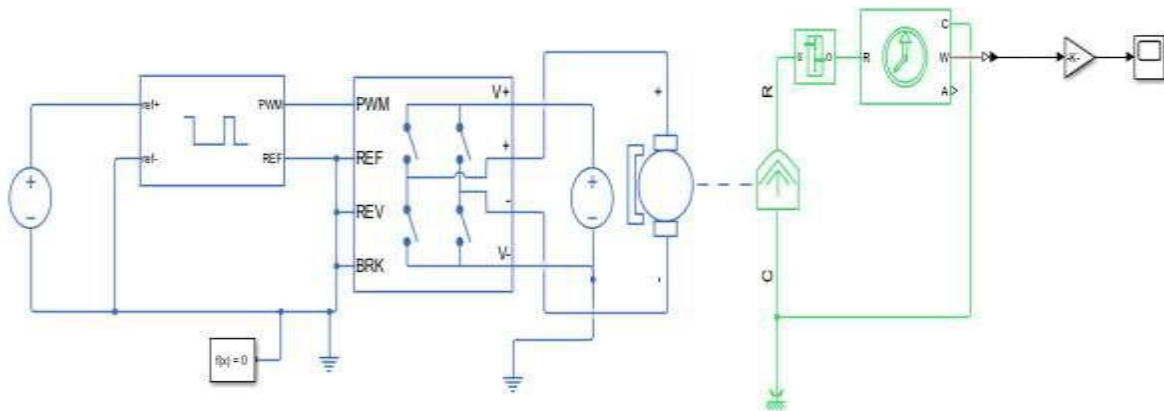


Figure 5: Circuit Diagram

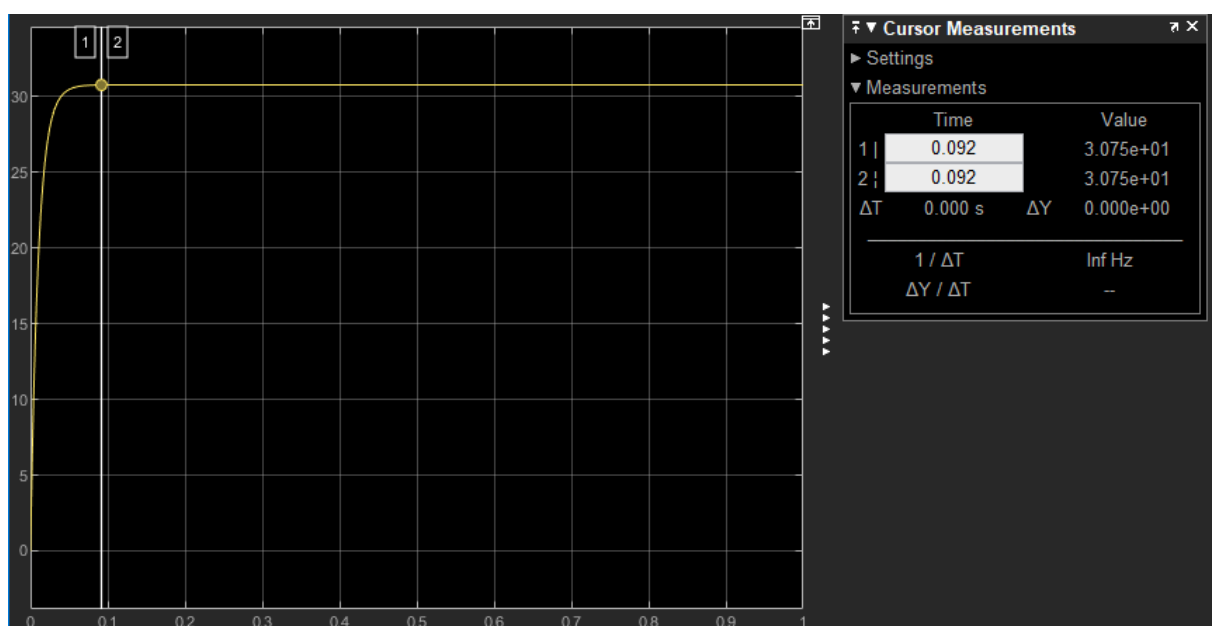


Figure 6: Scope Output of Diagram at 100 duty ratios

As one can see from Figure 6 at a 100-duty ratio, 7.5V, the theoretical Simulink diagram settles at a similar rpm of 30.76 to the actual 31. However, it has a much faster settling time than actual of 0.092 instead of 0.432. This is potentially due to two factors. The fact that this model ignores the inherent inductance of the motor as well as the fact that the theoretical model does not account for the friction of the gears, and only the motor's viscous friction.

One can still make the motor behave in a similar way to the actual in that one may reverse the direction or slow the speed. To reverse the direction all we need to do is swap the motor connection's polarity as shown in Figure 7.

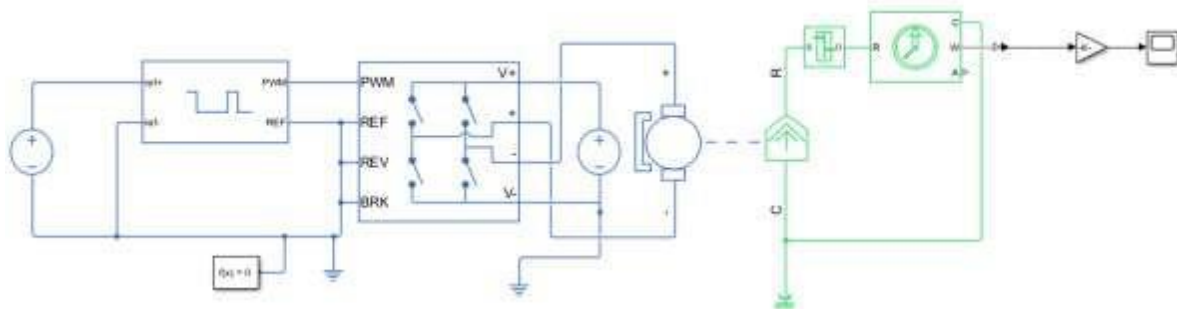


Figure 7: Reversed Direction Diagram

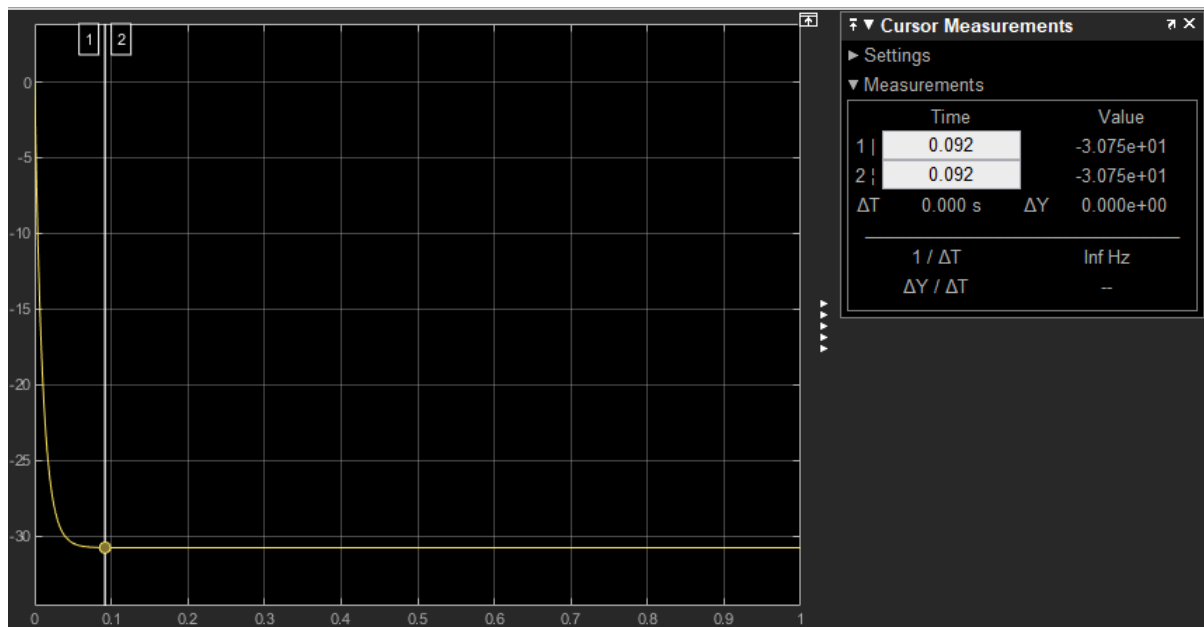


Figure 8: Reversed Scope

To slow the response all one needs to do is adjust the first DC power supply. Here it was changed from 100% to 50% by changing the 5V power supply to 2.5V. As seen in Figure 9, this produces a slower rise time and lower steady state speed compared to the 100% duty cycle. It is also both larger and faster than the real one, again most likely due to the frictional damping constant issues which are exacerbated at slower speeds.

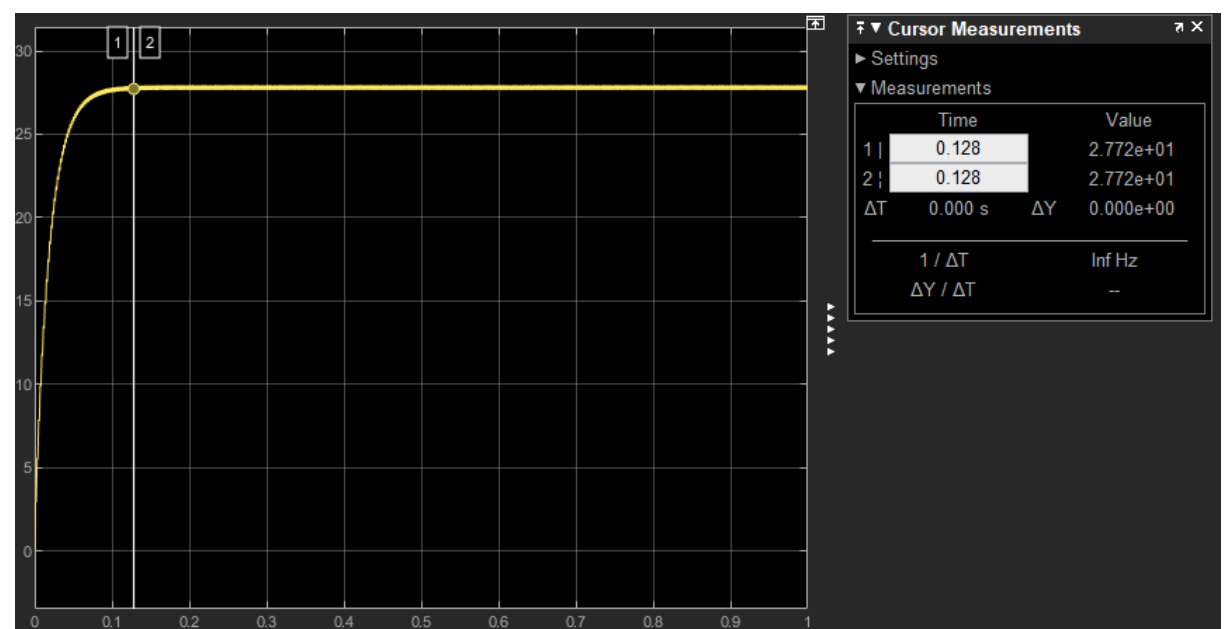


Figure 9: Slowed Scope

Position Control in Simulink Using Real Time Target

Here, the potentiometer position will set the reference position of the rotor shaft. When the potentiometer is moved from one position to another, the rotor shaft should move from one position to another.

Modelling and Simulations

In this setting, potentiometer position must be read by the Arduino and be set as a reference signal. The motor encoder is wired to the Arduino to help calculate rotor position in real time. Reference signals are then compared to encoder readings, and the difference between them (position tracking error) must be multiplied by a proportional controller gain K_p . The arising signal is the command to the motor but must be first mapped to an appropriate PWM value, which is then sent to the H-bridge to drive the motor. Adjusting the position tracker made in previous parts, the resulting model is shown in Figure 21.

Hardware Simulink Model

The potentiometer is an analog signal which ranges from 0-1023 so which should indeed be mapped from 0-255 for the Arduino board to interpret which gives us a gain value of 0.25. For position control, we need to map a 0-360° range to a 0-255 duty range, giving us a gain value of **0.7083**.

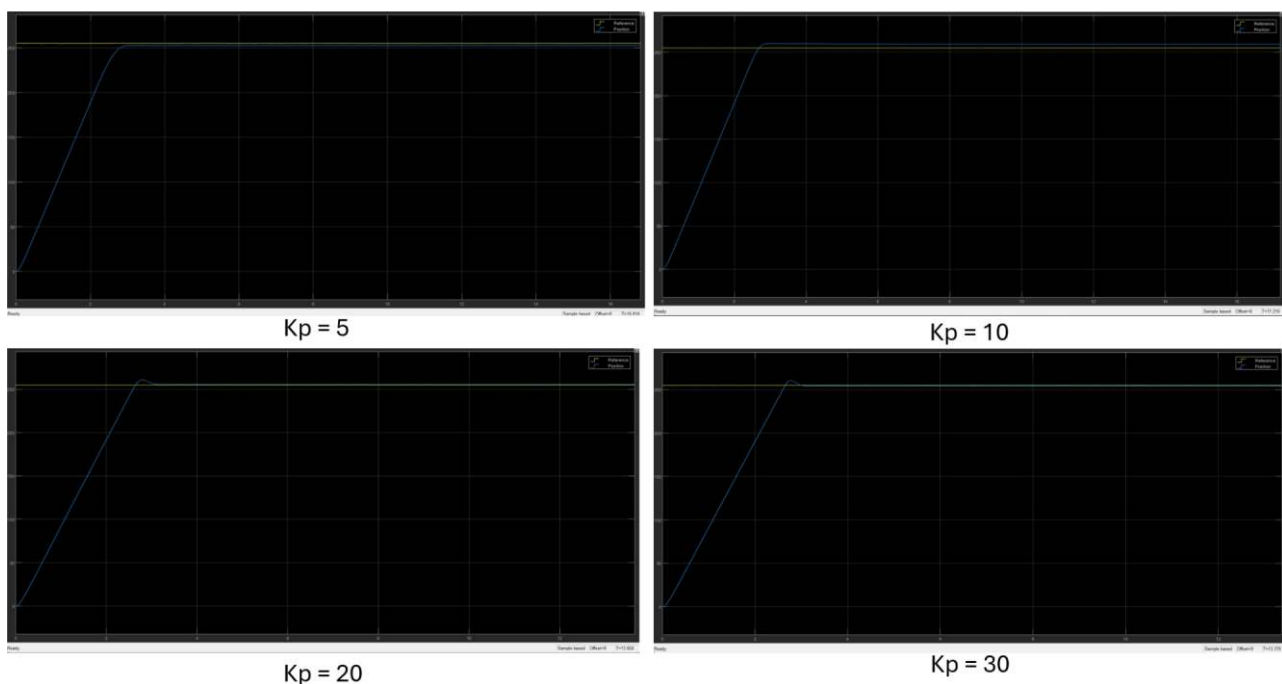


Figure 1: Examining K_p Values

This plot shows variation of the error as a function of time for different K_p values showing us that proportional gain of $K_p=20$ seems to give the least error without any overshoot.

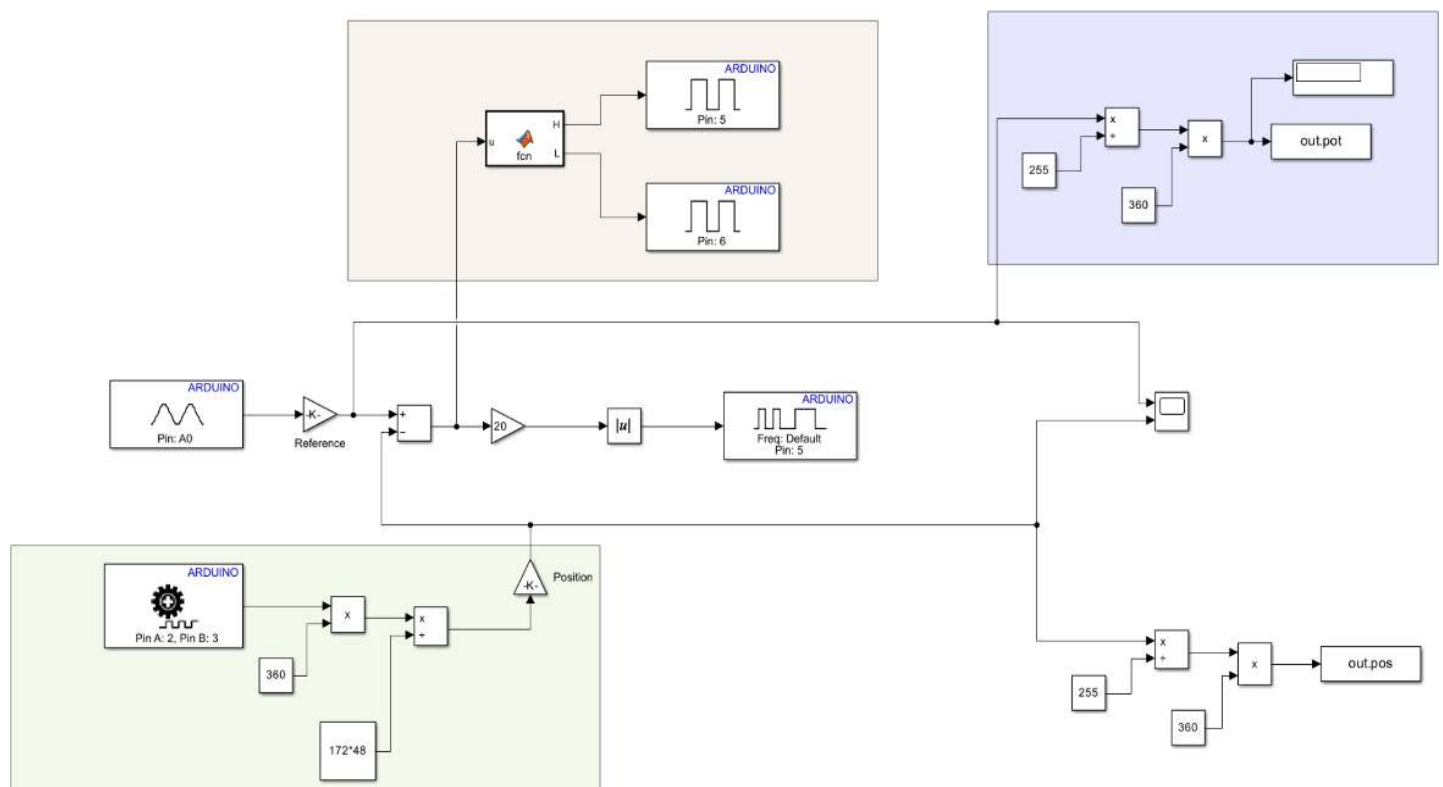


Figure 2: Position Simulink Model

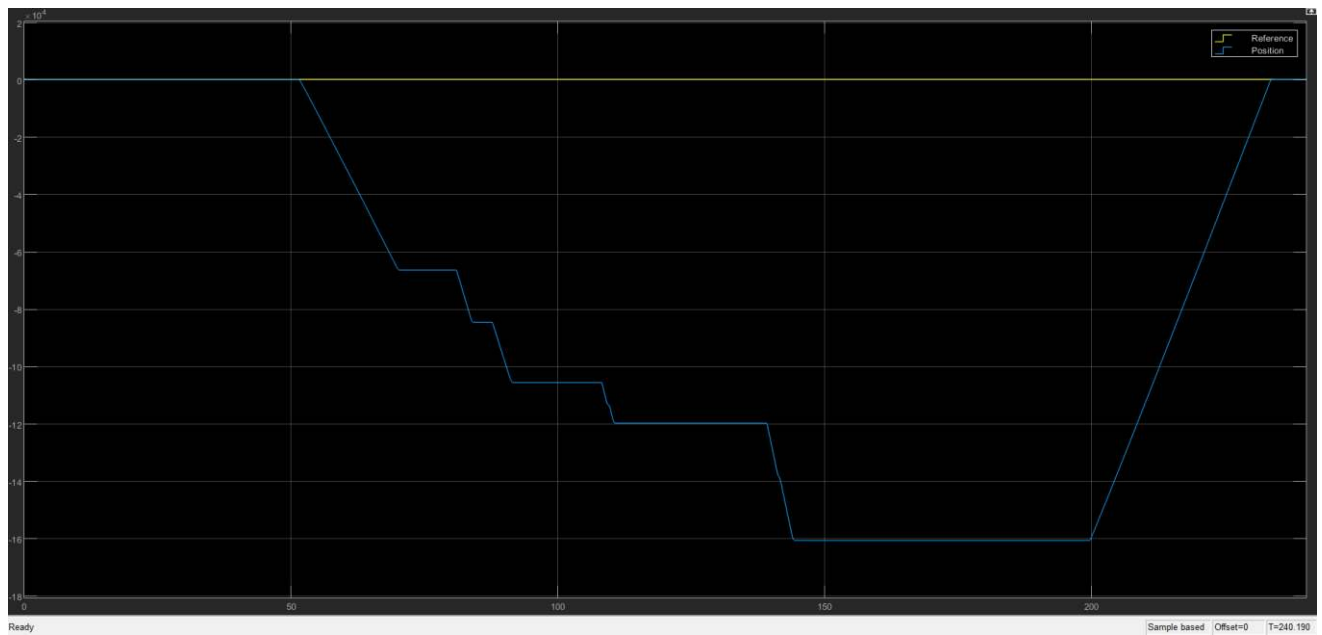


Figure 3: Position Tracking

Position Control in Simscape

he simscape model was developed to create the position control system as described in Control Objective. Here the position is adjusted in a closed loop, using a controlled voltage source, until the correct position is reached. The Simscape model for the feedback control system is integrated to meet Control Objective.

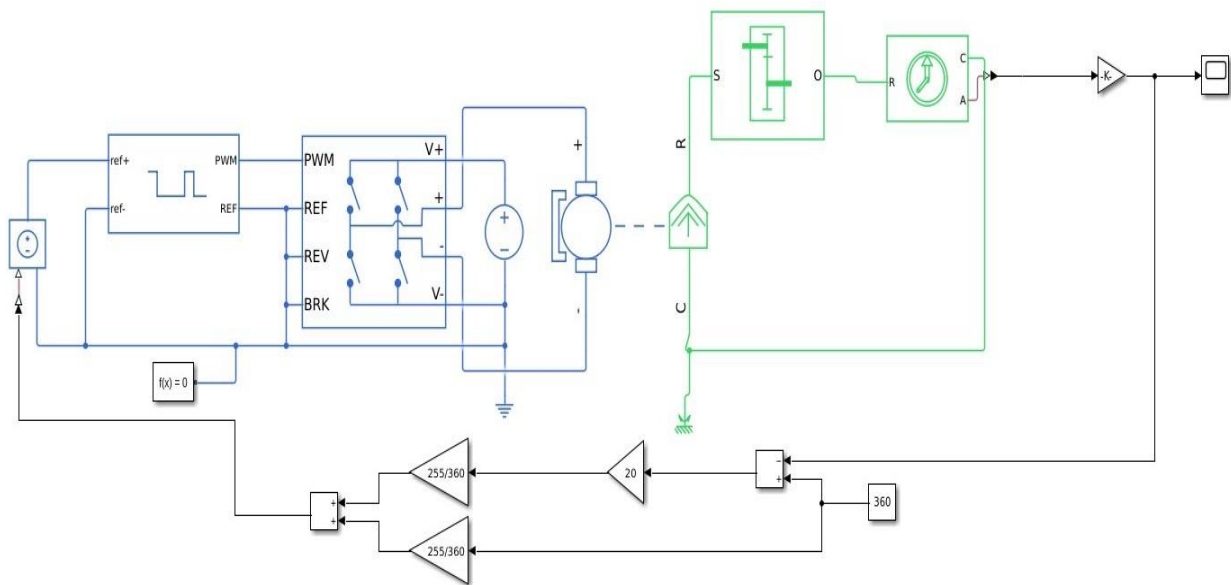


Figure 4:Simscape model of Position Control

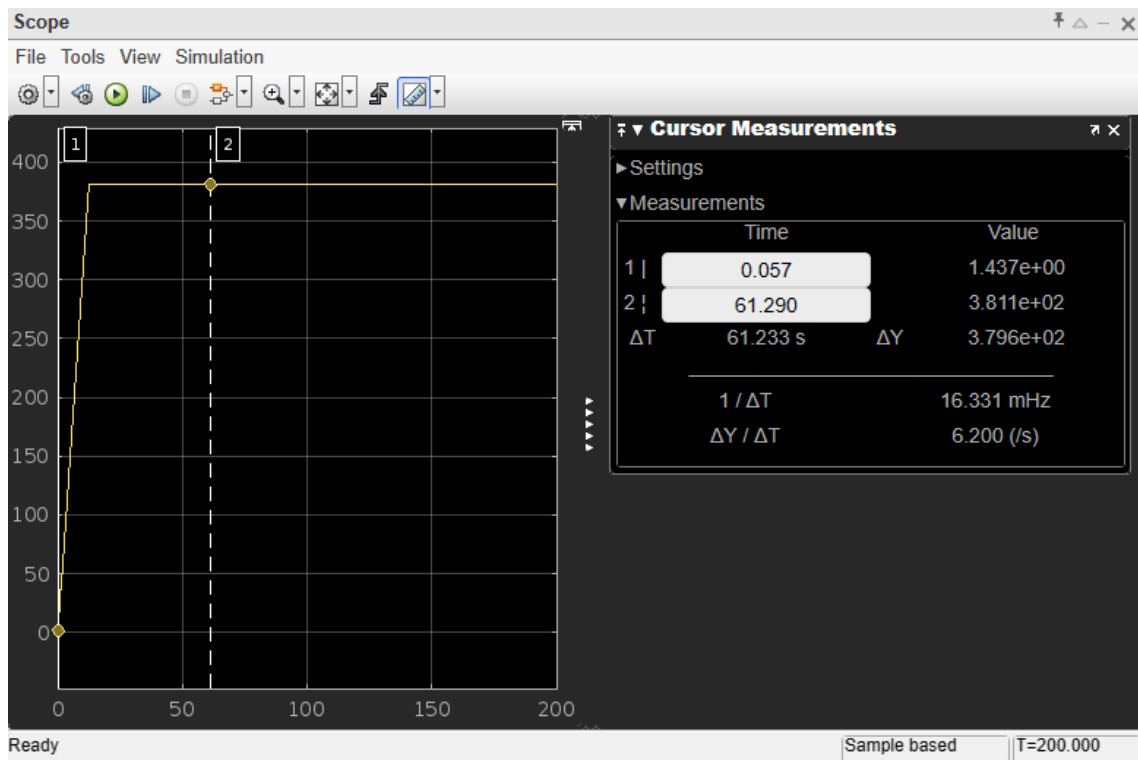


Figure 5:Position Simscape Scope

For a gain value of 20 we observe that the tracking is achieved, and the response of the system settles at a value close to the reference input set by the potentiometer which is 360 degrees

Speed Control in Simulink

Here, the potentiometer sets a speed from 0 to a maximum of 31.25 as a reference velocity. Moving the potentiometer there changes the reference velocity, increasing or decreasing the speed in one direction. The motor encoder is then wired to the Arduino, and in Simulink compares the reference speed to the actual speed. The velocity error is then multiplied by a proportional controller gain K_p . This then properly adjusts the PWM value until the appropriate speed is reached. Using the velocity tracker made in previous parts the resulting model is shown below in Figure 25.

Simulink Model

The final K_p value that ended up working best for the hardware model was 35. So 20, 35, and 50 were taken as the values of K_p and the Speed Vs Time graph is plotted. At $K_p = 20$ the graph had the actual speed not very close to the reference speed. When $K_p = 50$, while keeping the value close to the reference speed, introduced much oscillation into the graph. When $K_p = 35$, the graph showed the best of both with the actual speed very close to the reference speed, without an unexpected amount of error.

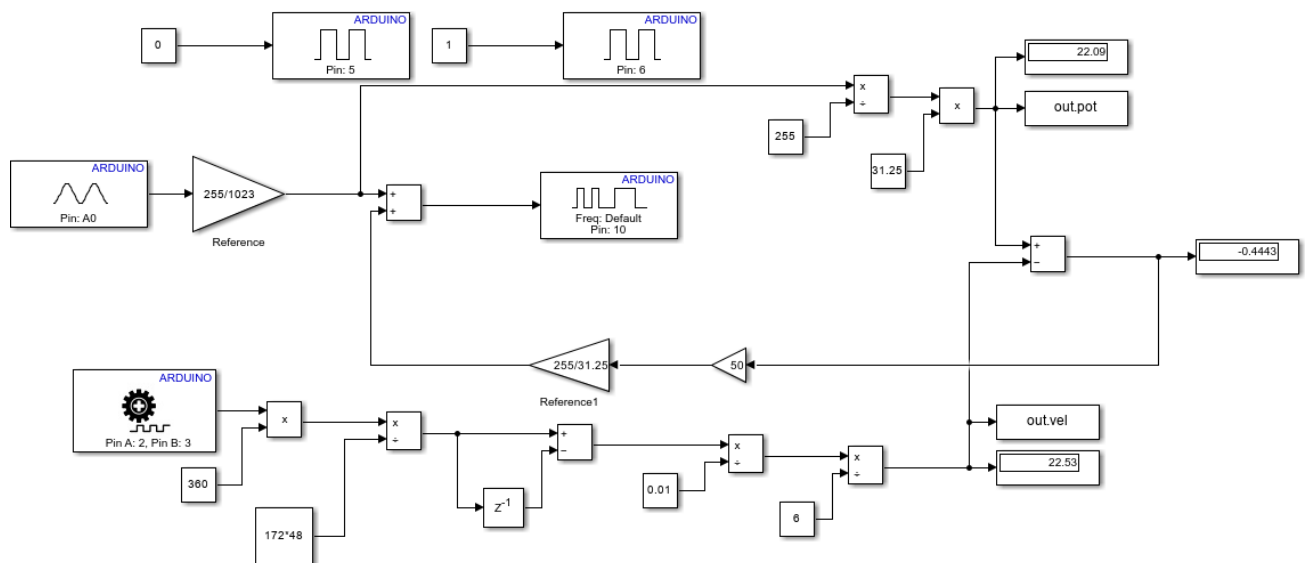


Figure 6: Simscape model of Velocity Control

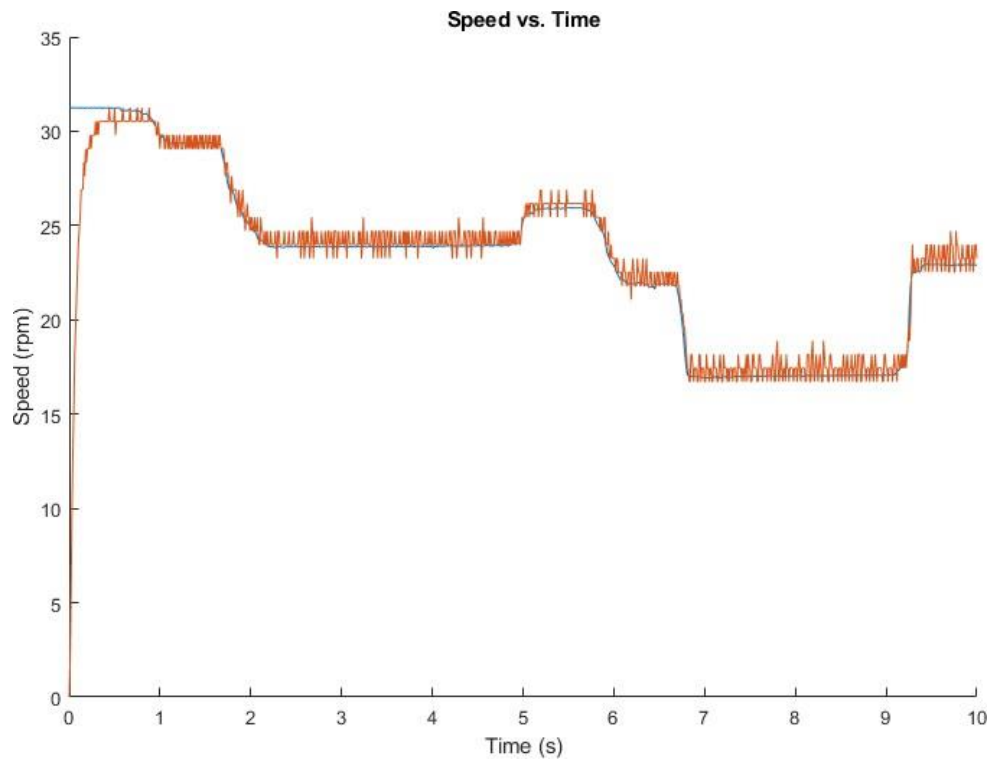


Figure 7: At $K_p = 20$

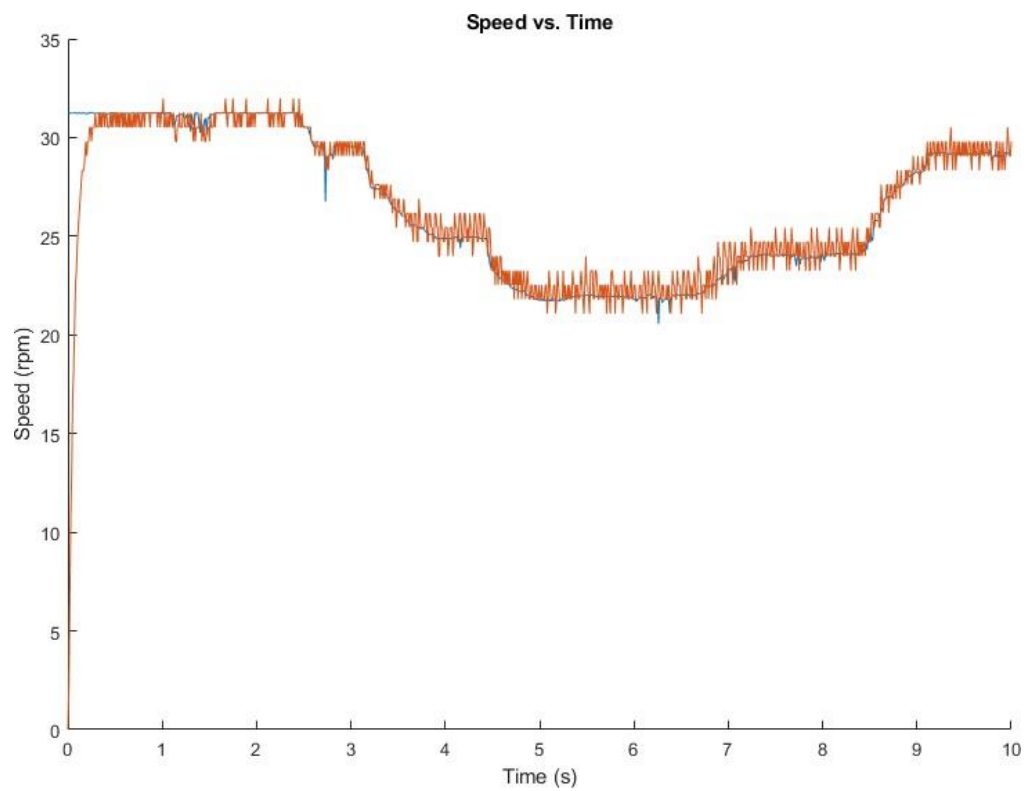


Figure 8: At $K_p = 35$

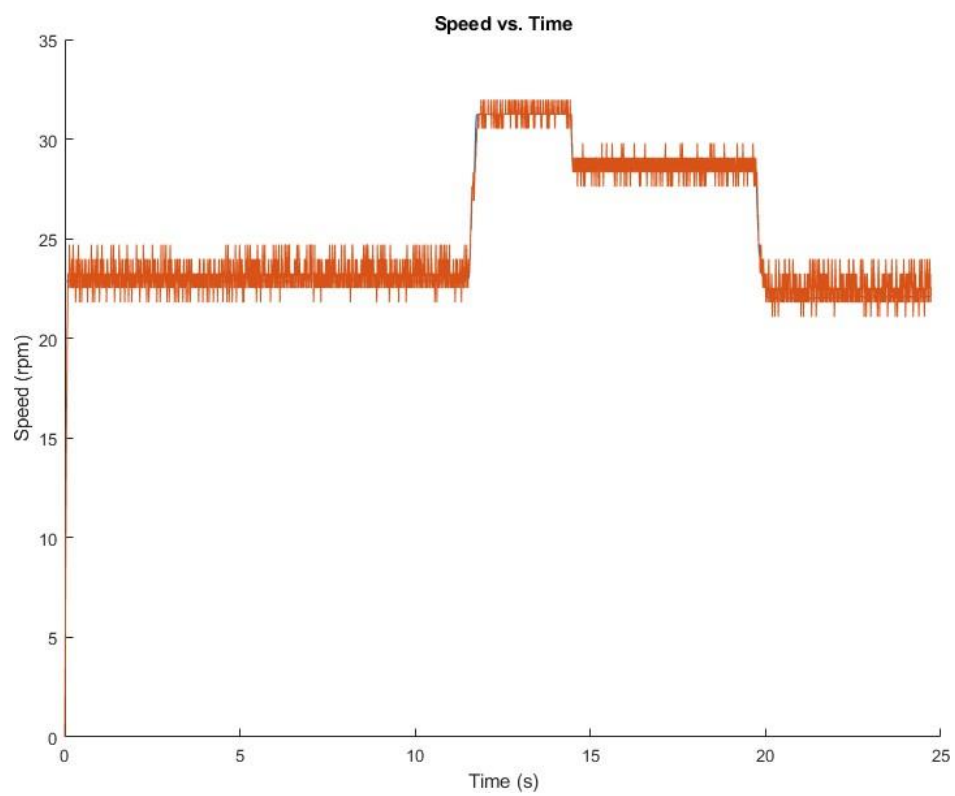


Figure 9: At $K_p = 50$

Speed Control in Simscape

The Simscape model for velocity is incredibly like the one for the for position with the only real difference being swapping the value for the maximum position for the maximum velocity. The value for this model is 30.76. The graph at different speeds and gains is generated. The speeds are at 30 and 20 rpm and the gain values are 20 and 40 at a speed of 20 rpm, with 30 being the ultimate gain chosen for the ideal velocity Simscape control model.

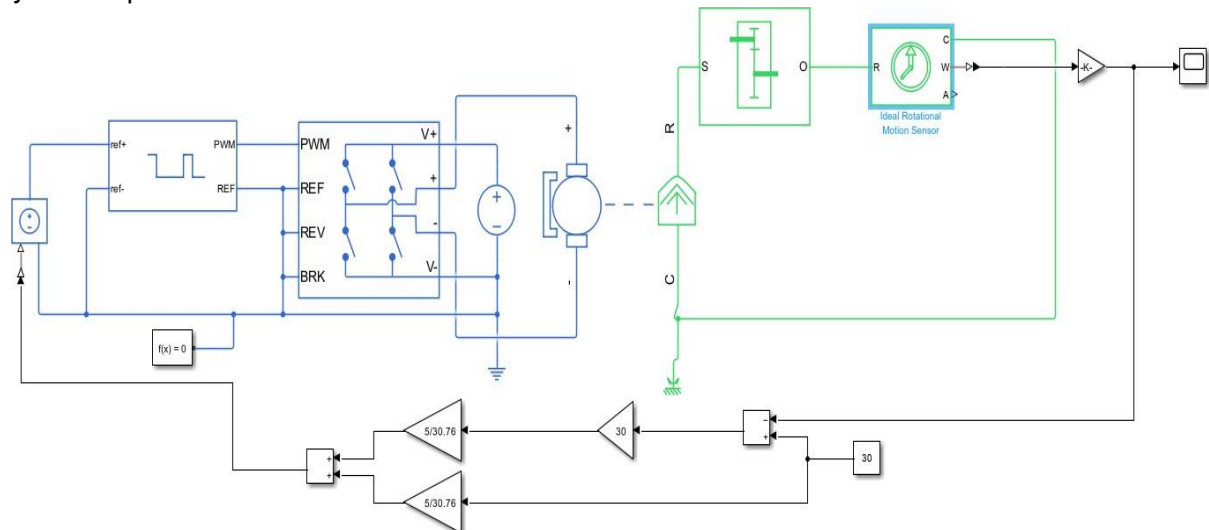


Figure 10:Simscape model of speed control

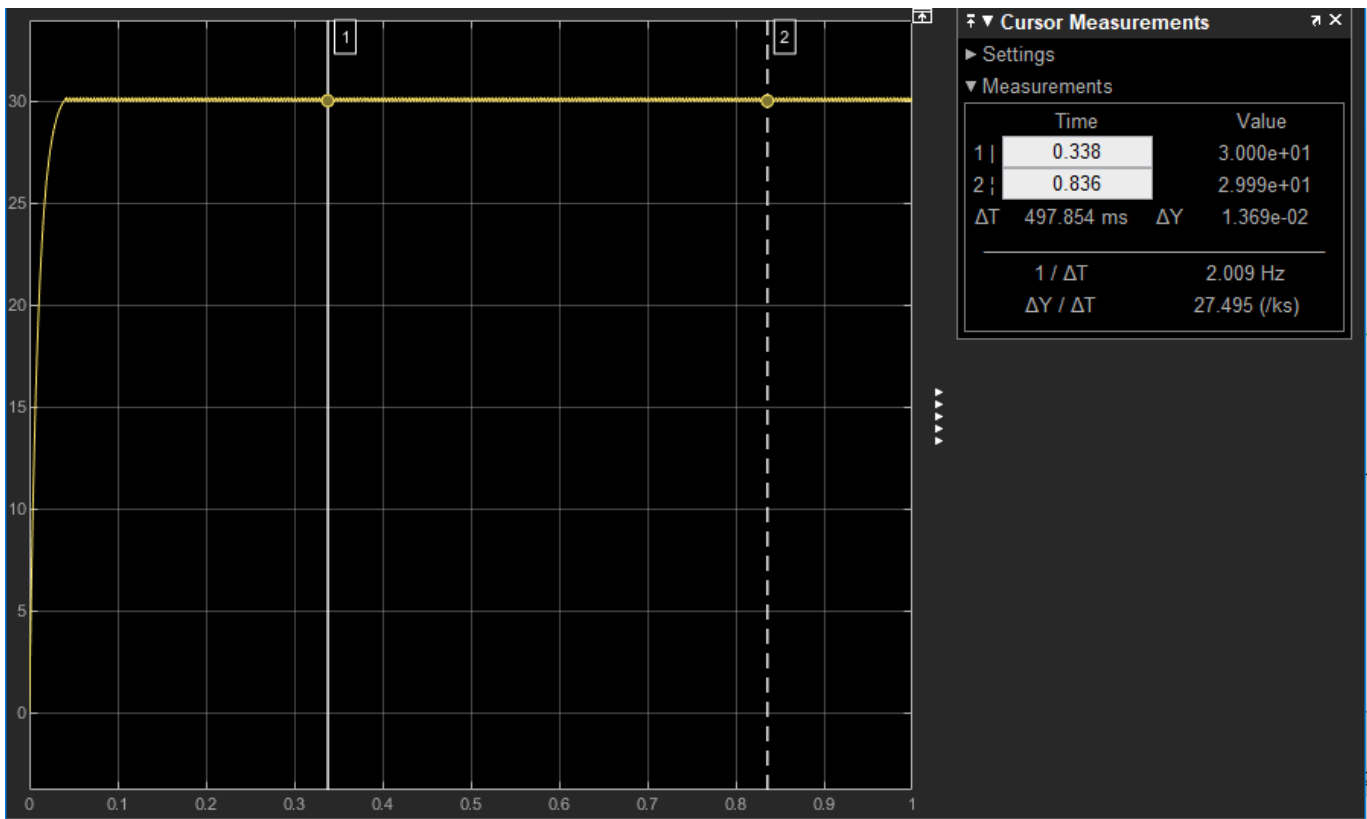


Figure 11: At Gain = 30 and Speed = 30rpm

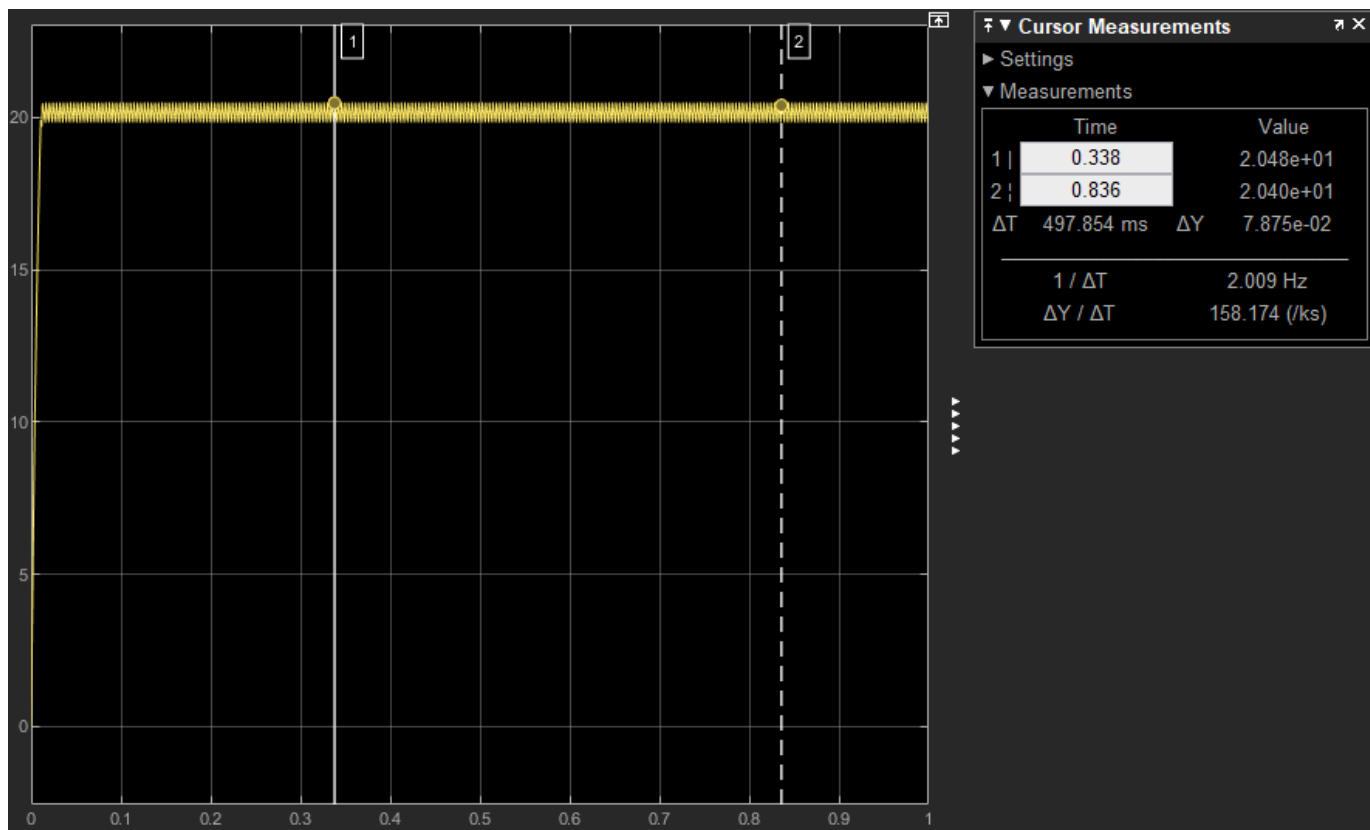


Figure 12: At Gain = 30 and Speed = 20rpm

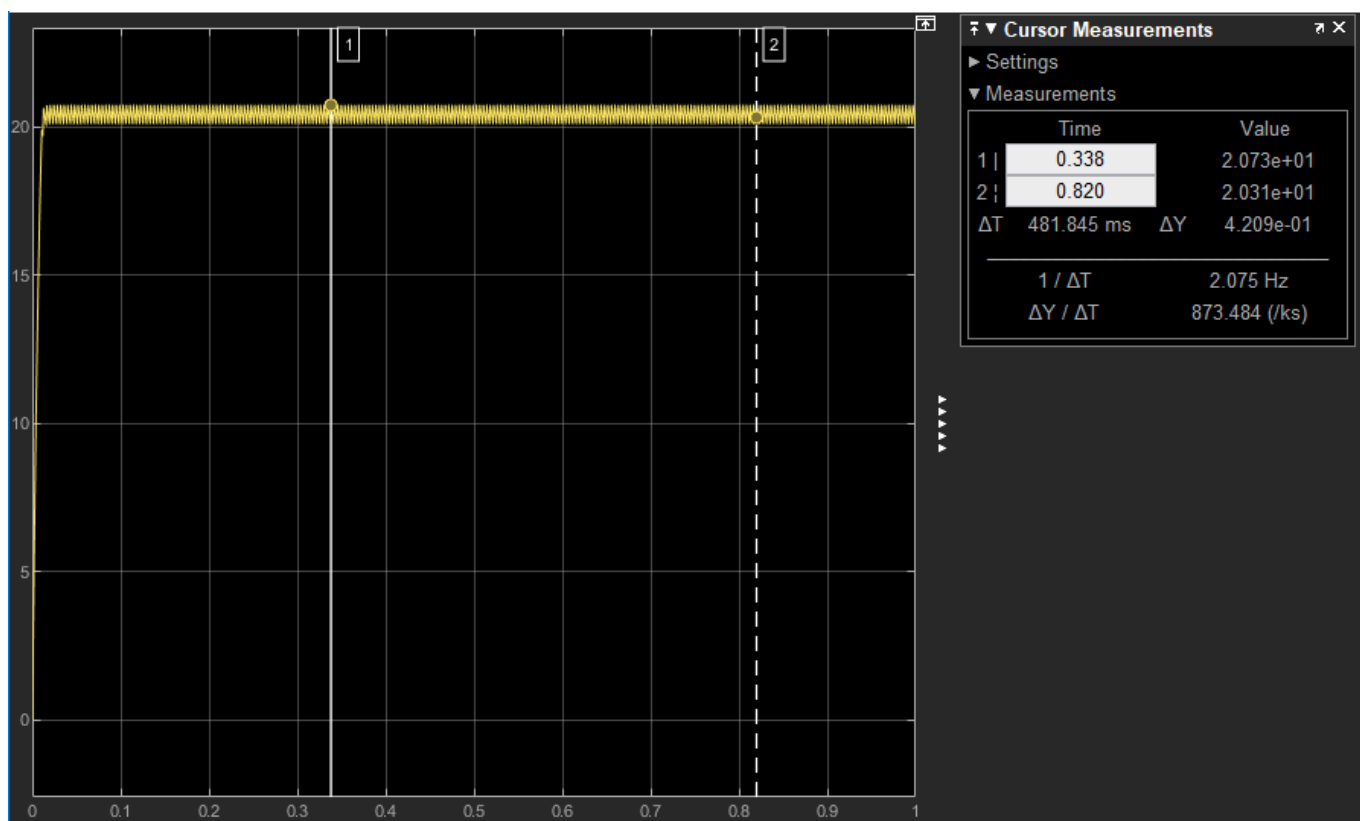


Figure 13: At Gain = 20 and Speed = 20rpm

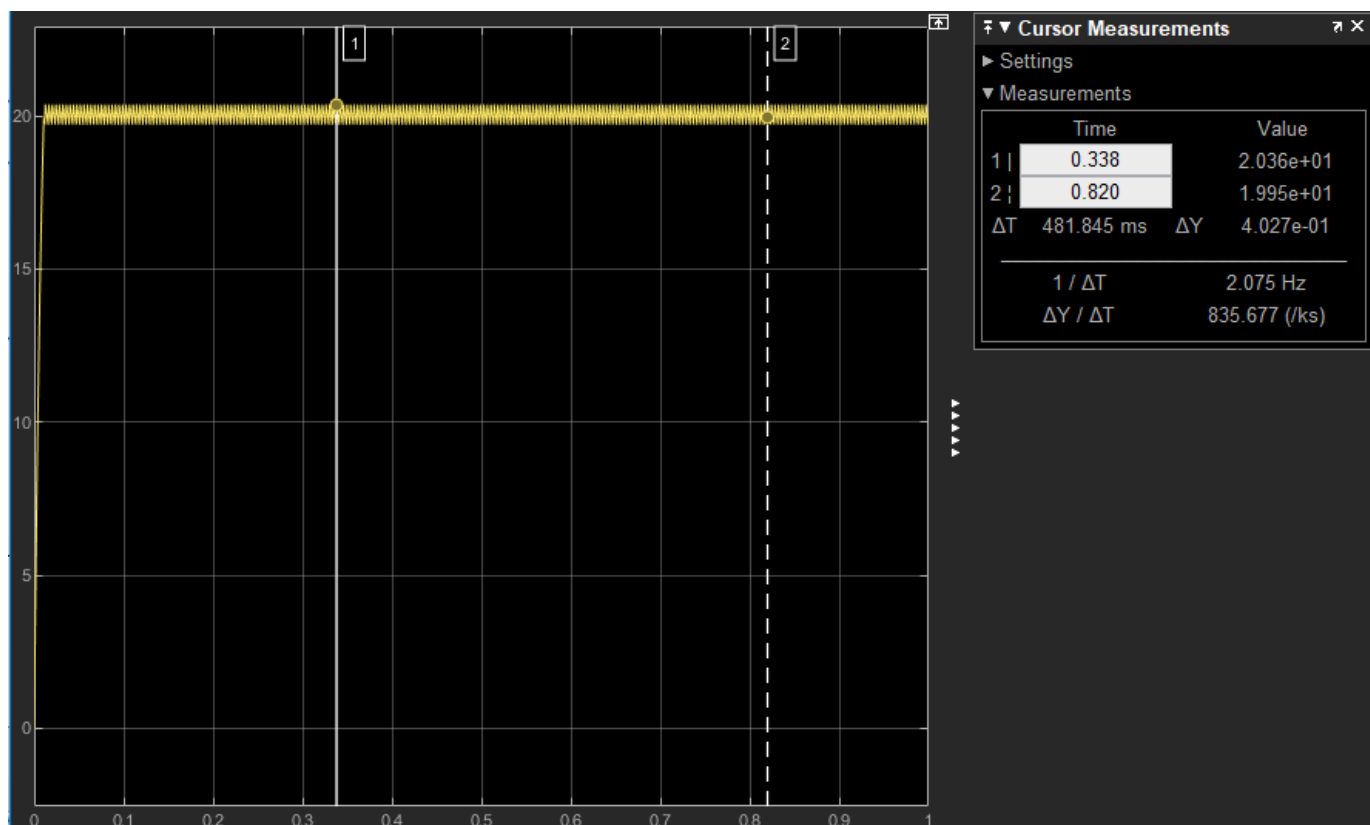
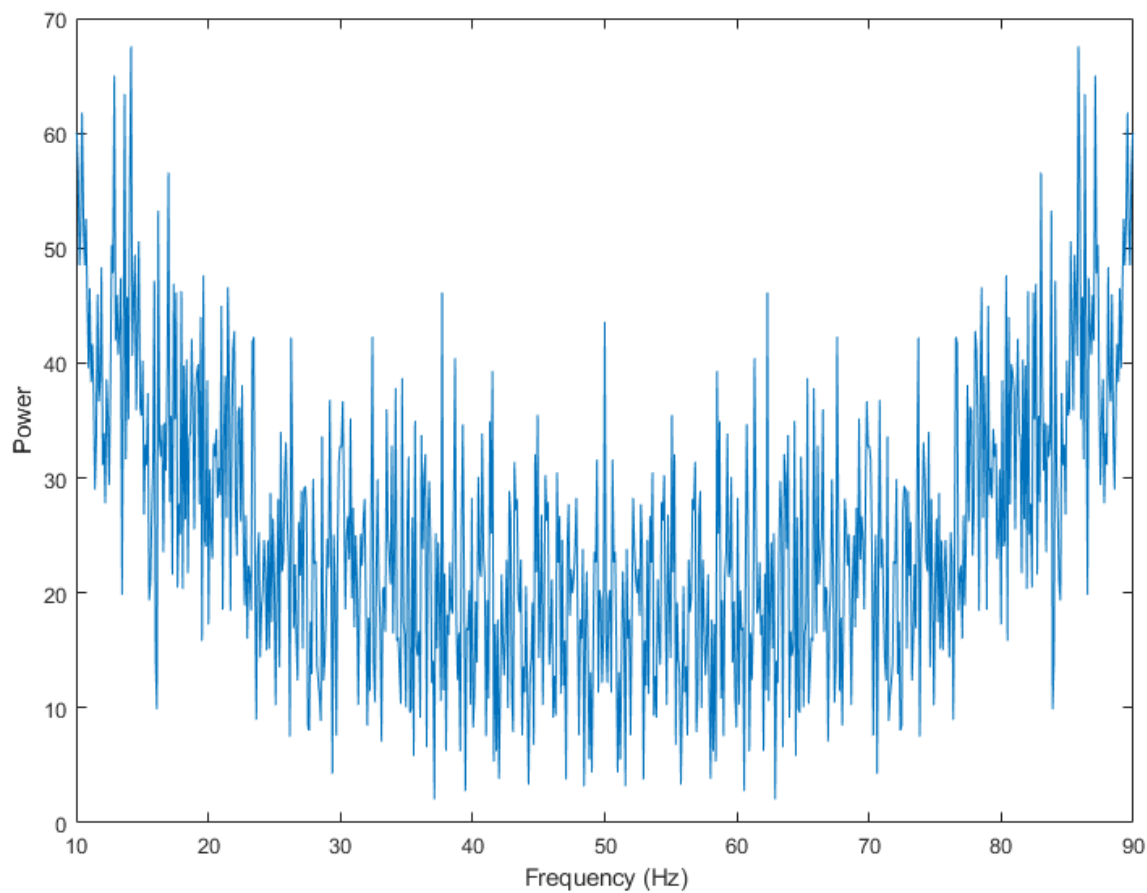


Figure 14: At Gain = 40 and Speed = 20rpm

When the gain is too low, the error does not properly reach the desired steady state speed, and when the gain is very high, the steady state error is introduced. At higher speeds there is less error than at lower speeds.

The simscape model has a much quicker rise time than the experimental model as expected. Also, the simscape model needed slightly less gain to appropriately reach its target velocity. But behaves very similarly when it comes to the k_p controller value and the error that it introduces. Like in the real model the lower the k_p value the more the output is off from the expected input. This can be seen best in the Speed vs Time graphs. However, too high of a k_p value and too much error and oscillation is introduced to the system. Another similar behavior is the further away from the maximum speed the reference is moved, the more error is introduced.

FFT at Maximum Motor Speed



FFT Graph